⚙️ B담당: 엔진 & 코드 작업 가이드

> **프로젝트:** 성수동 2D 픽셀 RPG
> **역할:** 동작하는 모든 것 (백엔드, 게임 엔진, 데이터 파이프라인)
> **기간:** 5일

네가 만들어야 할 최종 산출물

산출물	형식	설명
FastAPI 서버	Python	index.html 서빙 + 게임 상태 저장/로드 API
KAPLAY 게임 엔진	JS	맵 렌더링, 캐릭터 이동, 충돌, 카메라, 대화 시스템
OSM 데이터 추출 스크립트	Python	성수동 도로/건물 그리드 밑그림 생성 (A담당에게 전달)
네이버 API POI 수집	Python	옷가게 이름/좌표/카테고리 수집
DB 스키마 + 저장 API	Python	SQLite + SQLAlchemy ORM
배포 설정	설정파일	Render/Vercel 클라우드 배포

핵심 규칙: 너는 `.js`, `.py`, `.html`, `.css` 코드 파일만 건드린다. `views/assets/` 폴더의 PNG/JSON 에셋은 A담당 영역이다.

기술 스택 확정

계층	기술	이유
게임 엔진	KAPLAY (Kaboom.js)	함수형 구조, Cursor AI 환각 적음, 코드 간결
백엔드	Python FastAPI	비동기, Pydantic 데이터 검증 내장
DB	SQLite + SQLAlchemy	설정 비용 0, ORM으로 추후 PostgreSQL 전환 가능
맵 에디터 연동	Sprite Fusion JSON	A담당이 export한 JSON을 KAPLAY에서 로딩
지도 데이터	OSMnx (OpenStreetMap)	법적 안전, 벡터 데이터로 자동 그리드화
POI 데이터	네이버 Places API	옷가게 이름/좌표 수집용

인터페이스 계약 (A담당과의 약속)

Day 1 저녁에 A와 확정할 내용. **이게 흔들리면 Day 2 합류에서 터진다.**

■ 타일 크기: 16x16 px (또는 32x32 — A와 합의)
■ 맵 JSON: Sprite Fusion 기본 export 포맷
■ 에셋 파일 위치: views/assets/ 하위
■ 에셋 파일명 규칙: tileset_[용도].png
■ NPC 대사 JSON 구조:

```
{
  "npcs": [
    {
      "id": "shop_001",
      "name": "가게이름",
      "position": { "x": 타일좌표, "y": 타일좌표 },
      "sprite": "npc_shopkeeper_01.png",
      "lines": ["대사1", "대사2", "대사3"]
    }
  ]
}
```

Day 1: 프로젝트 뼈대 + 데이터 파이프라인

오전 (3~4시간): 프로젝트 구조 + 백엔드 기초

1단계: 폴더 트리 생성

```
seongsu_rpg_project/
├── core/
│   ├── config.py
│   ├── logger.py
│   └── exceptions.py
├── models/
│   ├── schemas.py
│   └── database.py
├── services/
│   ├── game_logic.py
│   └── api_routes.py
├── views/
│   ├── index.html
│   ├── style.css
│   ├── game.js
│   └── assets/          ← A담당 에셋이 들어갈 폴더
├── tests/
│   ├── test_logic.py
│   └── test_api.py
├── main.py
└── requirements.txt
```

2단계: Python 가상환경 + 의존성 설치

```bash
cd seongsu_rpg_project
python -m venv venv
source venv/bin/activate       # Windows: venv\Scripts\activate

pip install fastapi uvicorn sqlalchemy pydantic
pip install osmnx geopandas    # OSM 데이터 추출용
pip install requests           # 네이버 API 호출용

pip freeze > requirements.txt
```

3단계: main.py — FastAPI 최소 서버

목표: `localhost:8000` 접속 시 `views/index.html`이 뜨는 것까지만 확인.

Cursor AI 프롬프트 예시:

```
"FastAPI를 사용해서 views/ 폴더를 정적 파일로 마운트하고,
루트 경로(/)에서 views/index.html을 서빙하는 최소 코드를 작성해줘.
uvicorn으로 실행 가능하게."
```

확인 방법:

```bash
uvicorn main:app --reload
# 브라우저에서 localhost:8000 → 빈 페이지 또는 "Hello" 텍스트 확인
```

오후 (3~4시간): OSM 데이터 추출 + 네이버 API

4단계: OSM 데이터 추출 스크립트

목표: 성수동 옷가게 밀집 지역(연무장길~서울숲길 일대)의 도로/건물 벡터 데이터를 추출하고, A담당에게 줄 밑그림 이미지를 생성한다.

Cursor AI 프롬프트 예시:

```
"OSMnx를 사용해서 서울 성수동 연무장길 일대
(대략 중심 좌표: 37.5445, 127.0565, 반경 300m)의
건물 footprint와 도로 네트워크를 추출하고,
matplotlib으로 시각화해서 PNG 이미지로 저장하는 Python 스크립트를 작성해줘.
건물은 회색, 도로는 검은색, 배경은 흰색으로."
```

산출물:

- `seongsu_osm_grid.png` → A담당에게 전달 (Sprite Fusion 밑그림용)
- `seongsu_buildings.geojson` → 추후 타일 그리드 변환에 사용 가능

> ⚠️ **OSMnx가 1시간 이상 막히면 과감히 포기.** 구글맵에서 성수동 옷가게 구역 스크린샷 찍어서 A에게 밑그림으로 전달. 핵심은 "A가 내일 맵을 그릴 수 있는 참고 자료"를 주는 것.

5단계: 네이버 Places API로 POI 수집

목표: 성수동 일대 옷가게/편집숍의 이름, 좌표, 카테고리를 JSON으로 수집.

사전 준비:

1. 네이버 클라우드 플랫폼 가입
2. Application 등록 → Client ID / Client Secret 발급
3. 서비스: Maps > Places (검색 API)

Cursor AI 프롬프트 예시:

```
"네이버 지역 검색 API(v1/search/local.json)를 사용해서
'성수동 옷가게', '성수동 편집숍', '성수동 빈티지' 키워드로
각각 검색하고, 결과를 합쳐서 중복 제거 후
{name, address, mapx, mapy, category} 형태의 JSON으로 저장하는
Python 스크립트를 작성해줘."
```

산출물:

- `seongsu_poi.json` → 게임 내 NPC/가게 배치 데이터로 활용
- A담당에게도 공유 (어떤 가게를 넣을지 참고)

> 💡 네이버 API도 막히면? 수동으로 10개 정도만 구글맵에서 가게 이름+주소를 JSON에 직접 입력. 자동화는 보너스이지 필수가 아님.

Day 1 마감 체크리스트

- [ ] 폴더 구조 완성
- [ ] `pip install` 완료, `requirements.txt` 생성
- [ ] `main.py` → `localhost:8000`에서 index.html 서빙 확인
- [ ] OSM 밑그림 이미지 생성 (또는 대체 스크린샷)
- [ ] 네이버 API POI 수집 (또는 수동 10개)

⭐ Day 1 저녁 합류 (A담당과 10분)

A에게 전달:

- `seongsu_osm_grid.png` (밑그림)
- `seongsu_poi.json` (가게 목록 — 어떤 NPC를 만들지 참고용)

A와 확정:

- **타일 크기** (16x16 vs 32x32)
- **파일명 규칙** (위 인터페이스 계약 참조)
- **NPC 대사 JSON 구조** 확정

Day 2: KAPLAY 엔진 세팅 + 맵 로딩 시스템

종일 작업: 임시 맵으로 엔진 완성

핵심: A가 실제 맵을 만드는 동안, B는 **더미 맵(5×5 타일)**으로 엔진 코드를 완성한다. Day 2 저녁에 더미를 실제 맵으로 교체.

1단계: KAPLAY 엔진 로딩

`views/index.html`에 KAPLAY CDN 스크립트 태그 추가. `views/game.js`에서 KAPLAY 초기화.

Cursor AI 프롬프트 예시:

"KAPLAY(Kaboom.js) CDN을 사용해서
views/index.html에 캔버스를 설정하고,
views/game.js에서 kaplay()로 엔진을 초기화하는 코드를 작성해줘.
캔버스 크기는 브라우저 전체 화면(fullscreen).
배경색은 검정."

2단계: 더미 타일맵 로딩 + 렌더링

5×5 크기의 간단한 더미 JSON 맵 데이터를 만들고, 이를 화면에 렌더링.

Cursor AI 프롬프트 예시:

"KAPLAY에서 아래 구조의 JSON 타일맵을 로딩하고 캔버스에 렌더링하는 코드를 작성해줘.
타일 크기는 16x16.
각 타일 인덱스는 스프라이트 시트의 프레임 번호에 대응.
더미 데이터로 5x5 맵을 하드코딩해서 테스트."

3단계: 캐릭터 이동 + 충돌

Cursor AI 프롬프트 예시:

"KAPLAY에서 플레이어 캐릭터를 생성하고:
1. 화살표 키(또는 WASD)로 4방향 이동
2. 걷는 애니메이션 프레임 전환
3. 'wall' 태그가 있는 타일과 충돌 시 이동 차단 (area() + body() 사용)
이 기능을 views/game.js에 함수형 체이닝 방식으로 구현해줘."

4단계: 카메라 추적

"KAPLAY에서 카메라가 플레이어를 따라가도록 camPos()를 매 프레임 갱신하는 코드.
맵이 브라우저보다 클 때 스크롤되어야 함."

Day 2 마감 체크리스트

- [ ] KAPLAY 엔진 초기화 완료
- [ ] 더미 맵 렌더링 성공
- [ ] 캐릭터 4방향 이동 동작
- [ ] 벽 충돌 차단 동작
- [ ] 카메라 추적 동작

⭐ Day 2 저녁 합류 (프로젝트 성패 좌우 ⚡)

A에게 받을 파일:

```
views/assets/
├── seongsu_map.json
├── tileset_road.png
├── tileset_building.png
├── tileset_objects.png
└── tileset_custom.png (있으면)
```

즉시 할 일:

1. 더미 맵 코드를 실제 `seongsu_map.json` 로딩으로 교체
2. 타일셋 PNG 경로 연결
3. `localhost:8000`에서 **성수동 맵 위를 걸어다니는 화면** 확인

안 되면? 대부분 아래 3가지 문제:

- JSON 파싱 에러 → Sprite Fusion export 포맷과 KAPLAY 로딩 코드의 키 이름 불일치
- 타일 인덱스 불일치 → 타일셋 PNG의 프레임 번호와 JSON의 GID 매핑 확인
- 이미지 경로 에러 → `views/assets/` 하위 경로가 정확한지 확인

Day 3: 상호작용 시스템

오전: 카메라 + 맵 경계 처리

- 카메라가 맵 밖을 비추지 않도록 경계 클램핑
- 맵 전환 영역 (있다면) 처리

오후: NPC + 트리거 + 대화창

트리거 존 시스템

Cursor AI 프롬프트 예시:

```
"KAPLAY에서 특정 타일 좌표에 보이지 않는 트리거 오브젝트를 배치하고,
플레이어가 해당 오브젝트의 area()에 접촉한 상태에서
Space 키를 누르면 이벤트가 발생하는 시스템을 만들어줘.
이벤트는 콜백 함수로 처리."
```

대화창 UI

Cursor AI 프롬프트 예시:

"HTML/CSS로 화면 하단에 RPG 스타일 대화창 오버레이를 만들어줘.
- 반투명 검정 배경 박스
- 왼쪽에 NPC 이름 (하얀 글씨)
- 중앙에 대사 텍스트 (타자기 효과로 한 글자씩 출력)
- Space 누르면 다음 대사로 넘어감
- 마지막 대사 후 대화창 닫힘
views/style.css에 스타일, views/game.js에서 제어."

NPC 대사 JSON 로딩

A가 만든 `npc_dialogues.json`을 fetch()로 로딩하고 트리거 시스템과 연결.

"views/assets/npc_dialogues.json 파일을 fetch()로 로딩하고,
각 NPC의 position 좌표에 트리거를 자동 배치하는 코드.
Space 키 이벤트 시 해당 NPC의 lines 배열을 대화창에 순서대로 표시."

Day 3 마감 체크리스트

- [ ] 카메라 경계 처리 완료
- [ ] 트리거 존 시스템 동작
- [ ] 대화창 UI 구현 (타자기 효과)
- [ ] NPC JSON 로딩 + 대화 연결

⭐ Day 3 저녁 합류

A에게 받을 파일:

- 수정된 `seongsu_map.json` (소품 추가 버전)
- NPC 스프라이트 PNG들
- `npc_dialogues.json`

맵 교체 → NPC 배치 → **가게 앞에서 Space 누르면 대화 나오는지** 확인.

Day 4: 백엔드 데이터 연동

종일 작업: 저장/로드 시스템

models/database.py — ORM 테이블 정의

Cursor AI 프롬프트 예시:

"SQLAlchemy ORM으로 아래 테이블을 정의해줘 (SQLite 사용):
- GameState 테이블:
  id (Integer, PK, autoincrement)
  player_name (String)
  player_x (Integer) — 타일 좌표
  player_y (Integer) — 타일 좌표
  gold (Integer, default=0)
  inventory (JSON) — 아이템 리스트
  discovered_events (JSON) — 발견한 이벤트 ID 리스트
  saved_at (DateTime, auto)
database.py에 엔진 생성 + 세션 팩토리 + Base 포함."

models/schemas.py — Pydantic 검증

"Pydantic v2 BaseModel로 아래 스키마 정의:
- SaveGameRequest: player_name(str), player_x(int, ge=0), player_y(int, ge=0),
  gold(int, ge=0), inventory(list[str]), discovered_events(list[str])
- SaveGameResponse: id(int), saved_at(datetime), message(str)
- LoadGameResponse: 위 필드 전부 포함"

services/api_routes.py — REST API

"FastAPI Router로 아래 엔드포인트 구현:
POST /api/save — SaveGameRequest 받아서 DB 저장, SaveGameResponse 반환
GET /api/load/{save_id} — DB에서 로드, LoadGameResponse 반환
GET /api/saves — 저장 슬롯 목록 반환
DELETE /api/save/{save_id} — 저장 삭제
모두 models/database.py의 세션을 사용하고,
services/game_logic.py의 순수 함수를 호출하는 구조."

프론트엔드 fetch() 연결

"views/game.js에서:
1. 게임 내 메뉴에서 '저장' 버튼 → POST /api/save 호출 (현재 좌표/골드/인벤토리)
2. 게임 시작 시 '불러오기' → GET /api/saves로 목록 표시, 선택하면 GET /api/load/{id}
3. 응답 데이터로 플레이어 위치/상태 복원
fetch()로 비동기 호출, 덤 클라이언트 패턴 준수 (JS에서 직접 계산 금지)."

Day 4 마감 체크리스트

- [ ] SQLite DB 파일 자동 생성 확인
- [ ] POST /api/save → DB에 레코드 저장 확인
- [ ] GET /api/load/{id} → 정확한 데이터 반환 확인
- [ ] 프론트엔드 저장/불러오기 UI 동작

- [ ] A가 추가한 이벤트 JSON 연동 (선택)

Day 5: 통합 테스트 + 배포 (A와 종일 합류)

오전: 버그 수정

- A와 전체 플레이테스트
- 충돌 버그 → A가 Sprite Fusion에서 수정, 재export
- 코드 버그 → 즉시 수정
- 저장/로드 테스트

오후 초반: 클라우드 배포

Render.com 배포 (무료, 추천):

1. GitHub에 프로젝트 push
2. render.com 가입
3. New → Web Service → GitHub 레포 연결
4. 설정:
   - Build Command: `pip install -r requirements.txt`
   - Start Command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
5. Deploy

또는 Railway.app (대안):

- GitHub 연결 → 자동 감지 → 배포

오후 후반: 최종 확인

- 라이브 URL로 실제 접속 테스트
- 모바일 브라우저에서도 확인 (터치 조작은 5일 범위 밖, 표시만 되면 OK)
- 핫픽스 있으면 git push → 자동 재배포

Day 5 마감 체크리스트

- [ ] 전체 플레이테스트 통과
- [ ] 클라우드 배포 완료
- [ ] 라이브 URL에서 정상 접속 확인
- [ ] 저장/로드 라이브 환경에서 동작 확인
- [ ] Git 레포 정리, README 확인

Cursor AI 활용 팁

프롬프트 작성 원칙

1. **파일 경로를 명시해라**: "game.js에서" 가 아니라 "views/game.js에서"
2. **프레임워크를 명시해라**: "타일맵 로딩" 이 아니라 "KAPLAY의 addLevel()을 사용해서 타일맵 로딩"
3. **한 번에 하나만 요청해라**: "이동+충돌+카메라+대화창" ❌ → 각각 별도 요청 ✅
4. **기존 코드를 컨텍스트로 줘라**: "현재 game.js 코드는 이렇고, 여기에 카메라 추적을 추가해줘"

.cursorrules 설정 (프로젝트 루트에 생성)

```
프로젝트: 성수동 2D 픽셀 RPG (웹 기반 싱글플레이어)

기술 스택:
- 프론트엔드: KAPLAY (Kaboom.js), 바닐라 HTML/CSS/JS
- 백엔드: Python FastAPI + SQLAlchemy + SQLite
- 맵 데이터: Sprite Fusion JSON export 포맷

아키텍처 규칙:
- 3계층 분리: views(프론트) / services(로직) / models(데이터)
- views/game.js: 렌더링과 입력만. 비즈니스 로직 계산 절대 금지.
- services/game_logic.py: 순수 함수만. UI 코드 접근 금지.
- 덤 클라이언트 패턴: JS에서 직접 상태 변경 금지, fetch()로 백엔드 호출.

코드 스타일:
- KAPLAY: 함수형 체이닝 방식 (클래스 상속 사용 금지)
- FastAPI: async 함수 사용
- 타일 크기: 16x16 (또는 32x32 — 확정 후 수정)
- 에셋 경로: views/assets/ 하위
```

비상 상황 대응

상황	대응
OSMnx 설치/실행 실패	포기. 구글맵 스크린샷으로 A에게 밑그림 전달
네이버 API 키 발급 지연	수동으로 가게 10개 JSON 직접 작성
KAPLAY로 Sprite Fusion JSON 로딩 실패	KAPLAY의 addLevel() 대신 수동 타일 배치 코드로 전환. 또는 Phaser.js로 엔진 변경 (최후의 수단)
Day 2 저녁 맵 로딩 실패	패닉 금지. JSON 키 이름 대조부터. Sprite Fusion export 옵션 변경 시도

상황	대응
백엔드가 Day 4에 안 끝남	저장 기능을 LocalStorage로 임시 대체. 배포 후 업데이트
배포 실패	Render 대신 Python Anywhere(무료) 또는 로컬 ngrok 터널링

합류 일정 요약

시점	하는 일	필수 확인
Day 1 저녁	A에게 밑그림+POI 전달. 인터페이스 계약 확정	타일 크기, 파일명 규칙, NPC JSON 구조
Day 2 저녁 ⚡	A의 맵 JSON을 엔진에 로딩	**맵 위를 걸어다닐 수 있는가?**
Day 3 저녁	A의 NPC 에셋+대사를 게임에 통합	**가게 앞 Space → 대화창 뜨는가?**
Day 5 종일	통합 테스트 + 배포	**라이브 URL 작동하는가?**

가장 중요한 것

> **Day 2 저녁에 맵이 로딩되면 이 프로젝트는 성공한다.**
> 그 전까지는 더미 데이터로 엔진을 완벽하게 만드는 데 집중해라.
> 백엔드는 Day 4에 해도 된다. 엔진이 먼저다.
> LocalStorage 임시 저장으로도 게임은 돌아간다. 서버 저장은 보너스다.

일일 우선순위 (시간이 부족할 때 뭘 버릴지)

우선순위	기능	없으면?
1 (필수)	맵 렌더링 + 캐릭터 이동 + 충돌	게임 자체가 안 됨
2 (필수)	NPC 대화 시스템	RPG가 아니라 걷기 시뮬레이터
3 (중요)	카메라 추적	작은 맵이면 없어도 됨
4 (좋으면)	저장/로드 (백엔드)	LocalStorage로 대체 가능
5 (보너스)	클라우드 배포	localhost로 시연 가능
6 (보너스)	이벤트/퀘스트 시스템	Day 5 시간 남으면